

`fID.readlines([size])` reads the file contents in such a way that each line is a list item:

- if the optional parameter `size` is given, whole lines totalling approximately `size` bytes (possibly after rounding up to an internal buffer size) are read
- if `size` is not given, it goes up until EOF (end-of-file)

```
>>> fID = open("/path/to/file/words.txt", "r")
>>> fileData = fID.readlines()
>>> print(fileData)
['aa\n', ...      # first line in fID
'aah\n', ...      # second line in fID
'aahed\n', ...    # third line in fID
'aahing\n', ...   # fourth line in fID
'aahs\n', ...     # fifth line in fID
'aal\n', ...      # sixth line in fID
... and so on until ...
'zymurgy\n']      # last line in fID
```

Iterating Over a File

Finally, you can also read a file, line by line, by iterating over its handle with a very simple `for` loop.

```
>>> fID = open("/path/to/file/words.txt", "r")
>>> for line in fID:
...     print(line)
```

At each loop iteration, a new line from the text file is fetched into `line`. The `for` loop automatically stops when it reaches EOF.

Note: This is (possibly) the most efficient solution, especially when dealing with large text files since:

- it's fully automatic
- each line is read, processed and then discarded.

Final Remarks

It is customary to close the file after processing:

```
>>> fID = open("/path/to/file/words.txt", "r")
>>> # do something with fID
>>> fID.close()
```

If you do not close the file, Python automatically close it for you when the program ends. If you feel you may forget to close the file, use the `with` statement:

```
>>> with open("/path/to/file/words.txt", "r") as fID:
...     # do something with fID
```

and `fID` is automatically closed at the end of the `with` block.

Note: The ``with`` keyword is powerful. It can create context managers for several resources: scheduling threads for parallel processing, reading/writing data, exception handling and others. Yet it's quite an advanced topic.

As you may have noted, every line is treated as a string (regardless of its content).

In order to parse the content of each line, you have to use

- casting
- regular expressions
- string methods.

As the latter is concerned, notable examples include:

- `s.rstrip([chars])`: returns a copy of a string `s` with trailing characters removed. `[chars]` specifies the set of characters to be removed. It defaults to whitespaces and escape characters (e.g., `\n`)
- `s.lstrip([chars])`: same as above, but it treats leading characters
- `s.strip([chars])`: same as above, but it treats both leading and trailing characters
- `s.split([char])`: splits a string `s` over a given `[char]`. It defaults to whitespaces.

Writing Text Data

In order to write data to a text file you first have to open a file:

```
>>> fID = open("/path/to/file/myFile.txt", "w")
```

The `open()` function takes the following inputs:

1. the path on your disk where you file will be saved (i.e., `myFile.txt`);
2. the `"w"` flag, that means that file has to be opened in *write mode*. and returns
3. the file handle (i.e., `fID`).

Warning: If ``/path/to/file/myFile.txt`` already exists and you open it in write mode, all of its former contents are gone!

Writing some data to a text file is just as easy as doing:

```
>>> fID.write(<some content>)
```

For example:

```
>>> fID.write("This is the first line \n")
>>> fID.write("This is the second line \n")
>>> fID.close()
```

Note: `write()` argument must be a string. So make sure you cast your data to a string.

Note: You have to close the file especially when you open it in write mode to make sure that all your data are flushed to the disk.

You can also add new contents to an already-existing file by opening it in *append mode* (flag `"a"`):

```
>>> fID = open("/path/to/file/myFile.txt", "a")
```

If you open a file in append mode, any call to the `write()` method will write data to the end of the file. For example:

```
>>> fID.write("This is a new line \n")
>>> fID.write("This is another new line \n")
>>> fID.close()
```

Pickle Files

Text files are a great option for saving your variables in a human-readable form. But what happens when you have:

- very complex lists-of-lists (or list-of-tuples)?
- very complex dictionaries?
- instances of a user-defined class?

It can be quite hard to parse these heterogeneous data structures into a text file. Not to mention that in order to load them again, you have also to write a quite complex loader.

In this regard, Pickle files are a great option.

The **serialisation** process is a way to convert a data structure into a linear form that can be stored on disk or transmitted over a network.

The Python `pickle` module allows you to serialise objects in a **binary** format:

- ✓ the Pickle protocol allows to easily read (*unpickle*) and write (*pickle*) Python data objects with 1 line of code
- ✓ you can serialise the vast majority of Python objects, including:
 - booleans
 - integers, floats, complex numbers
 - strings
 - tuples, lists, dictionaries, sets (if they contain pickable objects)
 - instances of user-defined classes
- ✗ Pickle file contents are not human-readable: you can only read a Pickle file thanks to the `pickle` module in Python
- ✗ Pickle files are only Python-related: cannot be used in other programming languages
- ✗ the Pickle protocol changed across Python versions: unpickling a file that was pickled in a different version of Python may not always work properly

Writing pickles

In order to write a Pickle file, you must first `import` the `pickle` module:

```
>>> import pickle
```

Then, choose the object(s) that you want to save, e.g. `Lnew`:

```
>>> L = [1, 3, 10]
>>> Lnew = [ ]
>>> for i in range(len(L)):
...     Lnew.append(L[i] * 2)
```

and then, the magic happens:

```
>>> outfile = open('myfile.pkl', 'wb')
>>> pickle.dump(Lnew, outfile)
>>> outfile.close()
```

What's going on in the last code snippet?

1. we `open()` a file named `myfile.pkl` in `wb` mode (*write binary*) and let `outfile` be its file handle
2. we write `Lnew` to the file
3. we close the file

You can also save multiple objects by "packing" them into a list or tuple, e.g.:

```
>>> outfile = open('myfile2.pkl', 'wb')
>>> pickle.dump([Lnew, L], outfile)
>>> outfile.close()
```

Warning: When you read a file with a list/tuple that contains multiple items, you must remember their order!

Note: The ``wb`` flag is mandatory to tell Python to write a (serialised) binary file. Do not confuse ``w`` with ``wb``!

Note: The file can be saved also without extension (i.e., ``myfile`` instead of ``myfile.pkl``) and in principle you can also use the extension you prefer (i.e., ``myfile.pickle`` instead of ``myfile.pkl``): Python will still understand that's a Pickle file.

Reading pickles

In order to read a Pickle file, you must first `import` the `pickle` module:

```
>>> import pickle
```

and then, the magic happens:

```
>>> infile = open('myfile.pkl', 'rb')
>>> X = pickle.load(infile)
>>> infile.close()
```

What's going on in the last code snippet?

1. we `open()` a file named `myfile.pkl` in `rb` mode (*read binary*) and let `infile` be its file handle
2. we read the contents of `infile` into variable `X`
3. we close the file

Compressing Pickle Files

If you have to save large data structures, Pickle files may take a lot of space on your hard drive.

Workaround? **Compression**.

Compression is the art of encoding (hence, saving) data with fewer bits in order to save disk space.

Warning: Do not confuse serialisation with compression!

Since there is no free lunch, compression is driven by a trade-off: - saves disk space by encoding data with fewer bits - the encoding stage can be costly (i.e., in terms of running times)

Python in its standard library provides two well-known compression algorithms:

- *gzip*, in the `gzip` module
- *bzip2*, in the `bz2` module

Again, since there is no free lunch, compression algorithms themselves do have some trade-offs:

- *bzip2* is slower than *gzip*
- *gzip* yields files with larger size

For the sake of example, let us consider *bzip2* (although similar interfaces hold for *gzip*).

Writing a Compressed Pickle File

In order to write a compressed Pickle file with *bzip2*, you must `import` both modules:

```
>>> import pickle
>>> import bz2
```

Let's consider a big list:

```
>>> fID = open("words.txt", "r")
>>> fileData = fID.readlines() # fileData is a list with 113783 words
>>> fID.close() # we don't need words.txt anymore
```

Now, let's save the list with `pickle`:

```
>>> pickle.dump(fileData, open("words.pkl", "wb"))
```

Now, let's save the list with a `bz2`-compressed `pickle`:

```
>>> bz2file = bz2.BZ2File("words.bz2", "w")
>>> pickle.dump(fileData, bz2file)
>>> bz2file.close()
```

What's going on in the last code snippet?

1. from the `bz2` module, we call `BZ2File` to create a file called `words.bz2` in *write mode* (flag `w`). This yields a file handle named `bz2file`.
2. we use the already-known `pickle.dump()` to save our variable `fileData`. However, instead of passing as argument a file handle obtained via `open()`, we pass the file handle obtained via `bz2.BZ2File()`
3. we close the `bzip2` file.

Result?

- `words.pkl` takes 1.2MB (=1200KB) on disk
- `words.bz2` takes 403KB on disk

The `bz2` file takes 1/3rd of the space with respect to the original `pkl` file.

Reading a Compressed Pickle File

In order to read a compressed Pickle file with *bzip2*, you must `import` both modules:

```
>>> import pickle
>>> import bz2
```

and then:

```
>>> bz2file = bz2.BZ2File("words.bz2", "r")
>>> X = pickle.load(bz2file)
>>> bz2file.close()
```

What's going on in the last code snippet?

1. from the `bz2` module, we call `BZ2File` to read a file called `words.bz2` in *read mode* (flag `r`). This yields a file handle named `bz2file`.
2. we use the already-known `pickle.load()` to import the contents of `bz2file` into a variable `X`. However, instead of passing as argument a file handle obtained via `open()`, we pass the file handle obtained via `bz2.BZ2File()`
3. we close the `bzip2` file.

References

- Gutttag, J. V. (2013). *Introduction to Computation and Programming using Python (Revised and Expanded Edition)*. MIT Press. [Section 4.6]
- Downey, A. (2015). *Think Python*. Green Tea Press ed. 2nd. [Sections 9.1, 14.1--14.3, 14.5, 14.7]
- <https://docs.python.org/3.9/library/pickle.html>
- <https://docs.python.org/3.9/library/bz2.html>
- <https://docs.python.org/3.9/library/gzip.html>
- <https://raw.githubusercontent.com/AllenDowney/ThinkPython2/master/code/words.txt>
- <https://www.datacamp.com/community/tutorials/pickle-python-tutorial>

